ExitPass v1.2 Reading Map / Guide

Purpose of this guide

This guide tells the student team exactly what to read, in what order, and how deeply. It is written for a team assigned to a bounded ExitPass slice centered on the Android application that will run on the MoPS device.

The team is not being asked to build all of ExitPass. They are being asked to understand the parts of the ExitPass v1.2 baseline that define their scope, their boundaries, their interfaces, and the rules they are not allowed to violate.

The assigned student scope

The student team should treat the project scope as follows:

- Build the Android application that will run on the MoPS device.
- Support continuity-mode operations through the MoPS device app.
- Support capture of continuity transaction data and required operational evidence.
- Support device-side workflows for incident tagging, manual-operation support, and operator actions if these are part of the assigned slice.
- Support handoff of captured records into the ExitPass continuity and reconciliation flow.
- Integrate with the working Vendor PMS adapter that will already be provided to them.

The student team is not responsible for building the Vendor PMS adapter. They are also not responsible for full payment orchestration, provider callback verification, platform payment finality, exit authorization issuance, or live gate-open authority.

How the team should read the ExitPass v1.2 baseline

The team should not read every document from start to finish in one pass. The correct order is:

1. BRD v1.2
2. SDD v1.2
3. API Contract Pack v1.2
4. Database Design v1.2
5. Engineering Pack v1.2
6. HikCentral OpenAPI guide, only for surrounding context

That order matters.

- The BRD explains why the subsystem exists and what business rules govern it.
- The SDD explains where the subsystem sits in the architecture.
- The API Contract Pack explains how services are supposed to talk.
- The Database Design explains where the data belongs and what boundaries must not be crossed.
- The Engineering Pack explains how implementation is supposed to behave.
- The HikCentral guide is only there to help the team understand the vendor-side context behind the already-provided adapter.

Reading intensity legend

Use this legend for every document.

- **Read deeply** means required reading for design and implementation.
- **Read selectively** means read only the subsections relevant to the assigned slice.
- **Skim only** means understand context but do not spend design effort here.
- **Reference only** means do not study unless a specific issue requires it.

---

1. ExitPass BRD v1.2

Status

The exact BRD section numbers and headings must be inserted after the BRD v1.2 file is uploaded into this workspace.

What to read deeply

The team should read deeply the BRD sections covering:

- project overview, purpose, and scope
- business objectives and system context
- authority model
- MoPS device role in continuity operations
- site group, site, and lane operating context
- business continuity or degraded-mode rules
- MoPS behavior and handling
- reconciliation requirements
- manual gate, incident, override, and exception handling
- assumptions, constraints, and exclusions

- non-functional requirements relevant to auditability, traceability, fail-closed behavior, idempotency, security, and access control

What to read selectively

The team should read selectively any BRD subsections that define:

- what data the MoPS device app captures
- what validations or prompts happen on the device side
- continuity activation or continuity usage criteria
- MoPS import or continuity evidence handling
- reconciliation outputs and exception closure rules
- app-side operator and incident support behavior

What to skim only

The team should skim, not study deeply:

- coupon and merchant wallet business flows
- statutory discount business rules, unless later assigned
- parker-facing WebPay details beyond context
- broad platform reporting outside continuity and reconciliation
- Vendor PMS adapter implementation details beyond interface context

What is reference only

The team should treat these as reference only:

- payment-provider-specific business rules
- full exit authorization behavior outside their assigned boundaries
- features clearly outside the student subsystem

Required output after BRD reading

After BRD reading, the team should be able to answer:

- What exact business problem is our subsystem solving?
- What belongs to Vendor PMS authority?
- What belongs to Central PMS authority?
- What is a continuity record in ExitPass, and what is it not?
- Why is reconciliation required after continuity operations?
- What is outside our implementation scope?

---

2. ExitPass SDD v1.2

Status

The exact SDD section numbers and headings must be inserted after the SDD v1.2 file is uploaded into this workspace.

What to read deeply

The team should read deeply the SDD sections covering:

- overall solution architecture
- runtime services and service ownership
- trust boundaries
- Central PMS and Vendor PMS separation
- the place of the MoPS device app in the overall architecture
- continuity-mode workflow
- MoPS workflow
- reconciliation workflow
- sequence diagrams involving continuity-mode usage, MoPS record capture, backend submission, review, and reconciliation
- architectural constraints that prevent the device app from becoming authoritative

What to read selectively

The team should read selectively any SDD subsections covering:

- app-to-backend interactions
- device authentication and trust binding
- data flow for continuity and reconciliation
- service interactions involving operations, audit, and incident handling

What to skim only

The team should skim:

- full payment-orchestrator internals
- provider callback verification details
- platform-wide observability and deployment architecture beyond awareness level

- full vendor adapter implementation internals

What is reference only

The team should leave as reference only:

- production infrastructure details they are not expected to build
- deep internal design of slices not assigned to them

Required output after SDD reading

After SDD reading, the team should be able to answer:

- Where does the MoPS Android app sit in the architecture?
- Which backend services does it interact with?
- Which service owns continuity records?
- Which service owns reconciliation records?
- Which service must never be bypassed?
- What sequence happens when the site enters continuity mode?

---

3. ExitPass API Contract Pack v1.2

Read deeply

The team should read these sections deeply:

Section 2. v1.2 contract baseline and source hierarchy

This tells the team where the contract sits in the overall document hierarchy and confirms that the BRD is the source of business scope and continuity expectations. This section is important because it stops the team from treating the API contract as if it can override the BRD or architecture.

Section 4. Locked contract position

This is mandatory reading. It states the non-negotiable rules the team must preserve, including:

- Vendor PMS remains the authority for raw parking session lifecycle and tariff computation.

- Central PMS must not become an independent tariff engine.
- Only Central PMS may finalize payment-linked state and issue exit authorization.
- MoPS and manual gate activity remain continuity and exception evidence.

The Android MoPS app must fit inside those rules. It is a continuity tool, not an authority replacement.

Section 5. Service ownership and ingress map

The team must understand which route families belong to which service, especially:

- `/v1/ops/*`
- `/v1/audit/*`
- any device-facing or operations-facing endpoints that the MoPS app will call

They only need `/v1/vendor/*` for context, because the vendor adapter will already be provided to them. This section tells them where their subsystem fits and where it does not fit.

Section 6. Common contract rules

Read especially:

- **6.2 Canonical headers**
- **6.3 Canonical error envelope**
- **6.4 Canonical identifiers**

These sections matter because the app's backend interactions must still behave like ExitPass APIs.

Section 7. Storage-enforced guarantees

Read especially:

- **7.1 Constraint-to-API error mapping**
- **7.2 Mandatory DB routine mapping**

This is where the team learns that API behavior is not free-form and that persistence rules govern outcomes.

Section 8. Status value contract

The team must use the defined status values and not invent their own lifecycle statuses in device or backend-facing flows.

Section 9. Endpoint contract catalog

This is the most relevant operational section for the student scope. The team must focus on these endpoint entries:

- `POST /v1/ops/manual-gate-logs`
- `POST /v1/ops/mops-transactions/import`
- `POST /v1/ops/reconciliation/runs`
- `PATCH /v1/ops/reconciliation/items/{reconciliation_item_id}`
- `GET /v1/audit/events`
- `GET /v1/audit/evidence/{evidence_link_id}`

They should read the vendor endpoints only for context so they understand the system around the MoPS app, not because they are implementing those endpoints.

Read selectively

Read selectively any detailed public API sections that directly support their workflow context, but do not turn those into implementation scope unless assigned.

Skim only

Skim these areas for awareness only:

- public parker payment-attempt creation
- coupon endpoints
- merchant endpoints unrelated to continuity and reconciliation
- event replay administration
- vendor endpoints not directly surfaced to the device app

Reference only

Use as reference only:

- payment-provider callback endpoints
- full public payment lifecycle endpoints outside their assignment

Required output after reading the API Contract Pack

The team should extract:

- the route families the Android app will call or depend on
- required headers
- canonical status values they must reuse

- error envelope shape
- which DB-backed actions are relevant to their slice

---

4. ExitPass Database Design v1.2

Read deeply

The team should read these sections deeply:

Section 5. Design Principles

Read the full section, especially:

- **5.1 Authority Separation**
- **5.3 Database-Enforced Control for Critical Transitions**
- **5.5 Idempotency, Atomicity, and Concurrency Safety**
- **5.6 Traceability First**
- **5.8 Fail-Closed Control**
- **5.10 Operational Exceptions Without Financial Corruption**

These principles are foundational to their slice.

Section 11. Audit and Traceability Model

This matters because continuity and reconciliation work is evidence-heavy and operationally sensitive.

Section 12. Logical Data Model

Focus deeply on:

- **12.3 System-of-Record Boundaries**
- **12.4 Schema-Level Domain Overview**
- **12.11 Operations and Exception Domain**
- **12.12 Continuity and Reconciliation Domain**
- **12.13 Site Topology Domain**
- **12.15 Identity and Access Domain**, especially ServiceIdentity
- **12.16 Audit and Evidence Domain**
- **12.17 Integration Metadata Domain**, only for context around the provided adapter
- **12.19 Eventing Domain**, enough to understand outbox and event boundaries
- **12.20 Cross-Schema Relationship Summary**

- **12.22 Domain Boundary Notes**
- **12.23 Logical Model Design Goals**

These are the sections that show exactly where their data belongs and what cross-domain moves are prohibited.

Section 13. Physical Table Specifications

Read deeply only the tables relevant to their assigned subsystem:

- **13.7 Operations Schema**:
  - `operations.manual_gate_logs`
  - `operations.override_requests`
  - `operations.override_approvals`
  - `operations.incident_records`
  - `operations.operator_action_logs`
- **13.8 Continuity and Reconciliation Schema**:
  - `reconciliation.mops_transaction_records`
  - `reconciliation.reconciliation_runs`
  - `reconciliation.reconciliation_items`
  - `reconciliation.reconciliation_exceptions`
  - `reconciliation.settlement_comparison_records`
- **13.9 Sites Schema**:
  - `sites.site_groups`
  - `sites.sites`
  - `sites.lanes`
  - `sites.device_assignments`
- **13.11 Identity Schema**:
  - `identity.service_identities`
- **13.12 Audit Schema**:
  - `audit.audit_events`
  - `audit.audit_trail_entries`
  - `audit.evidence_links`
- **13.13 Integration Schema**, context only:
  - `integration.vendor_systems`
  - `integration.vendor_endpoints`
  - `integration.adapter_mappings`
  - `integration.integration_credential_references`
  - `integration.integration_health_records`
- **13.15 Eventing and Outbox Schema**, selectively:
  - `events.domain_events`

    - `events.outbox_events`
    - `events.event_publications`
    - `events.dead_letter_records`
    - `events.consumer_checkpoints`

Section 14. Core Integrity Rules

Read enough to understand the protected canonical chain they must not bypass.

Section 15. Concurrency and Idempotency Design

This matters because MoPS import and operational actions must not be duplicate-prone.

Section 16. Database Functions and Procedures

Read to understand what critical transitions are expected to be routine-backed.

Section 17. Outbox and Event Persistence

Read to understand event reliability expectations.

Read selectively

Read selectively:

- sessions domain, if the app needs to understand later reconciliation or recheck behavior
- partitioning strategy
- retention and archival
- performance notes
- appendices, except where a specific controlled code set is needed

Skim only

Skim these for awareness only:

- coupon domain
- statutory discount domain
- merchants domain
- broad whole-of-system ERD sections outside their slice

Reference only

Use as reference only:

- index inventory generation plans
- partition management plans
- simulation scenarios outside their domain

Required output after reading the Database Design

The team should produce:

- a list of relevant tables by domain
- a list of relevant status fields
- a map of cross-domain boundaries they must not violate
- a list of audit and evidence tables they must use or reference

---

5. ExitPass Engineering Pack v1.2

Read deeply

The team should read these sections deeply:

Section 2. Source baseline and hierarchy

This tells them how the document set is meant to be used and confirms that BRD, SDD, Database Design, DDL, and API Contract Pack are part of one controlled baseline.

Section 4. Locked engineering position

This is mandatory reading. It explains the engineering stance they are not allowed to violate, including the rule that MoPS and manual gate actions remain continuity and exception evidence.

Section 5. Engineering workstreams

The student team should focus on:

- **Workstream 7, Continuity and reconciliation**
- the device-side and operations-facing parts that interact with continuity handling

They should understand Workstream 3 only enough to know how their app fits beside the working Vendor PMS adapter already provided. They should understand Workstream 4 only enough to avoid crossing into live exit-control scope.

Section 6. Recommended build sequence

This helps the team understand that continuity and reconciliation come after the authoritative mainline model and are not substitutes for it.

Section 7. Runtime service implementation brief

Read deeply the entries for:

- Central PMS
- Session Service, only for surrounding context
- Site Integration Adapter, only for surrounding context
- Gate Integration Service, only for boundary awareness
- Audit and Event Service

The team needs to know where the Android MoPS app sits relative to these services.

Section 8. Contract-critical API implementation inventory

Focus on:

- `/v1/ops/mops/transactions`
- `/v1/ops/reconciliation/runs`
- `/v1/audit/*`
- any operations-facing endpoints the app will rely on

Read `/v1/public/sessions/resolve` and `/v1/public/tariff/quote` only as context if they help the team understand upstream behavior.

Section 10. Database-backed implementation rules

This section is critical. It teaches the team that workflows are orchestrated by services but governed by storage-enforced rules.

Section 11. State machines and lifecycle enforcement

Focus on:

- `ManualGateLog`
- `MoPSTransactionRecord`

- any status-bearing objects they directly create or update

Section 12. Event, audit, and outbox baseline

This matters because continuity, incident, and reconciliation workflows must still produce durable events and audit trails.

Section 13. Security, identity, and trust boundary implementation

Read especially the parts on:

- device-originated calls
- service-to-service trust
- evidence fields
- RBAC
- audit requirements

Section 14. Provider and vendor integration implementation rules

Read especially:

- **14.2 Vendor PMS adapters**

This section is relevant for context only. The team is not building the adapter, but they need to understand the rule that vendor-specific behavior stays behind that boundary and that the MoPS app must not try to take over vendor or Central PMS authority.

Section 16. Operations, MoPS, and reconciliation implementation

This is directly in scope and must be read deeply.

Section 17. Test strategy and release gates

Read enough to understand expected discipline.

Section 20. Code traceability and agent rules

This matters if the student team will generate code with AI assistance or must follow traceability conventions.

Read selectively

Read selectively:

- observability sections
- DevOps sections
- provider-specific payment integration sections

Skim only

Skim:

- suggested sprints outside their workstreams
- open engineering decisions unrelated to their slice

Reference only

Use as reference only:

- production-readiness details that are clearly beyond student scope

Required output after reading the Engineering Pack

The team should extract:

- the exact workstreams relevant to their project
- the runtime services they interact with
- the engineering rules they must preserve
- the lifecycle states they must model correctly
- the tests they are expected to think about, even if only at prototype depth

---

6. HikCentral Professional OpenAPI Developer Guide V3.1.0

Purpose of this document for the student team

This is not an ExitPass baseline document. It is a vendor-reference document that may help the team understand what the already-provided Vendor PMS adapter is abstracting.

The students should not treat HikCentral as their primary implementation target unless the Android MoPS app needs to reflect specific vendor-side concepts, labels, or operational data that come through the provided adapter.

Read deeply

Read these sections deeply only if the Android MoPS app needs to reflect vendor-side parking concepts exposed by the provided adapter.

Chapter 3. Protocol Summary

Read especially:

- **3.1 Request and Response Rule**
- **3.2 Signature and Authentication**

This is useful only if the team needs to understand how the provided adapter may communicate with HikCentral behind the scenes.

Chapter 4. Typical Applications

Focus on:

- **4.7 ANPR**
- **4.8 Parking Lot Application**

These sections help the team understand the vendor-side parking context.

Chapter 5. API Reference

Focus on:

- **5.8 Vehicle and Parking API**

Read these endpoint entries for context:

- `POST /artemis/api/pms/v1/crossRecords/page`
- `POST /artemis/api/vehicle/v1/parkinglot/list`
- `POST /artemis/api/vehicle/v1/parkinglot/passageway/record`
- `POST /artemis/api/vehicle/v1/parkingspace/record`
- `POST /artemis/api/vehicle/v1/parkingfee/calculate`
- `POST /artemis/api/vehicle/v1/parkingfee/confirm`

These are reference inputs for understanding the vendor-side parking model, not the Android team's main implementation scope.

Read selectively

Read selectively:

- **5.5 Alarm and Event API**, if the team wants to understand vendor events surfaced through the adapter
- **Appendix A.1 Event Message Format**, especially ANPR-related event formats
- **Appendix A.3 Event Types or Alarm Categories**, only if needed for context

Skim only

Skim:

- access control APIs
- visitor APIs
- video APIs
- other non-parking modules

Reference only

Use as reference only:

- installation and full platform setup sections
- unrelated modules such as attendance, canteen, education, digital signage, and broad video capabilities

Required output after reading the HikCentral guide

The team should extract:

- a short list of vendor-side concepts that may surface through the provided adapter
- any labels or fields the Android app needs to present consistently
- any assumptions they are making about vendor-side data because the adapter is already provided

---

Recommended reading schedule

Pass 1, Business and architecture

- BRD v1.2, relevant scope and authority sections
- SDD v1.2, relevant runtime and workflow sections

Pass 2, contracts and interfaces

- API Contract Pack v1.2, relevant route families and contract rules
- HikCentral guide, relevant parking and protocol sections only if needed for context

Pass 3, data and implementation rules

- Database Design v1.2, relevant domains and tables
- Engineering Pack v1.2, relevant workstreams, implementation rules, and testing stance

---

What the team must produce after reading

After working through this guide, the team should submit:

- a one-page scope summary
- a boundary and authority summary
- a list of relevant BRD requirements
- a list of relevant SDD flows and service boundaries
- a list of relevant API endpoints the Android app will consume
- a list of relevant tables and status fields that affect the app's workflows
- a device workflow proposal for continuity-mode operation
- a list of hard rules the app must not violate
- a list of open questions and ambiguities

Final instruction to the student team

Read with discipline. Do not confuse document coverage with implementation assignment. ExitPass is broader than the subsystem you are being asked to build. Your job is to understand the specific part assigned to you, fit it into the locked ExitPass v1.2 authority model, and avoid crossing into responsibilities owned by other services or other workstreams.

You are building the Android application for the MoPS device, not the Vendor PMS adapter. The adapter will already be provided. Your job is to make the MoPS device app behave correctly during continuity operations, capture the right records and evidence, hand them off properly into ExitPass continuity and reconciliation flows, and stay inside the authority boundaries defined by ExitPass.

If you understand the right sections deeply and the wrong sections lightly, your design will be sharper, your scope will stay controlled, and your work will be much more useful.